
Inductive Bias Meta-Learning with Generative Models

Anna Novokshonova
MIPT
Moscow, Russia
email@phystech.edu

Oleg Bakhteev
MIPT
Moscow, Russia
email@phystech.edu

Fedor Sobolevsky
MIPT
Moscow, Russia
email@phystech.edu

Muhammadsharif Nabiev
MIPT
Moscow, Russia
email@phystech.edu

Abstract

This paper investigates inductive bias in machine learning models. By inductive bias we mean the preference of a model for certain types of functions or data structures over others. To analyze the inductive bias of a fixed model, we consider a problem of finding data that this model can fit and generalize on particularly well. Previous work demonstrated that generating labels for a fixed dataset allows one to extract the inductive bias. Here, we extend this approach by proposing a method for generating full synthetic datasets. We train a generative model to produce datasets on which the target model achieves strong generalization performance. We test the proposed framework on CNNs and RNNs, and analyze obtained datasets for each model.

Keywords: inductive bias, meta-learning, generative models, CNN, RNN.

1 Introduction

Inductive bias plays a fundamental role in a model’s ability to generalize [1]. Without additional assumptions, any hypothesis that fits a finite training dataset is considered equally likely by the model, which can lead to poor generalization. Thus, effective generalization requires a set of prior assumptions about the data, and these assumptions constitute the model’s inductive bias. Understanding and exploiting inductive bias can be a powerful tool for enhancing model performance.

The term “inductive bias” does not have a single, strict definition. For instance, inductive bias can mean underlying assumptions about the domain and task or specific implementation choices that constrain or order the space of hypotheses [2]. In this paper, by inductive bias we mean a model’s preference for certain types of functions or data structures over others, expressed through better generalization performance. For example, convolutional neural networks (CNNs) are inductively biased towards data with local structure and translation invariance [3–5], while recurrent neural networks (RNNs) are inductively biased towards ordered data with sequential structure [6–8].

A clear understanding of the correspondence between models (or their specific components) and inductive biases is useful for solving machine learning problems. However, this task is highly non-trivial: for most models with complex architecture, the problem of detecting inductive bias is analytically intractable. Several approaches have been developed to extract and characterize the inductive bias of models. Boopathy et al. [9] proposed a method to quantify the amount of inductive bias of tasks or models as the amount of information required to specify well-generalizing models within a given hypothesis space. Si et al. [10] investigated the inductive bias of in-context learning

by constructing underspecified demonstrations. Li et al. [11] proposed a framework to capture the inductive biases in a learning system by meta-learning Gaussian process kernel hyperparameters from its predictions.

One recent result in this area is the meta-learning framework introduced by Dorrell et al. [12], which identifies the inductive bias of differentiable supervised learning algorithms by generating labels for a fixed dataset. For all its strengths, this approach has certain limitations. First, it requires access to input data or its distribution. Second, its analysis of inductive bias is constrained by the fixed dataset and its structure.

Here, we propose to extend this approach by generating entire synthetic datasets. The meta-learning loop proceeds as follows: an outer generative model, or meta-learner, produces a dataset, which is used to train and test an inner model, or learner, whose inductive bias we aim to extract. After the learner is trained, the quality of its predictions on test data from this dataset is incorporated into the meta-learner’s loss function. Thus, the meta-learner learns to generate datasets on which the target model shows the best generalization performance; these are datasets towards which it is inductively biased.

The proposed approach requires no knowledge of the input data distribution, making it more flexible. We expect it to enable a more comprehensive analysis of inductive bias.

Contributions. Our contributions can be summarized as follows:

- We present a generative meta-learning framework for extracting the inductive bias of a target model. Unlike prior work [12] that generates only labels for samples from a fixed distribution, our approach produces entire synthetic datasets via a meta-learned generative model.
- We empirically evaluate our algorithm on CNNs and RNNs and compare the results with the known inductive biases of these architectures. For CNNs, we expect to detect a local structure in the generated datasets; for RNNs, we expect them to have a sequential structure.
- We highlight the implications of our findings for understanding and leveraging inductive bias. Our method is general-purpose: it can be used to extract inductive bias of any differentiable supervised learning algorithm and, consequently, to improve the generalization performance of models in machine learning problems.

Outline. The rest of the paper is organized as follows. First, we formally state the problem we address and describe the proposed method in detail. Next, we validate it through experiments with CNNs and RNNs, analyzing the obtained datasets for the well known inductive biases of these architectures. Finally, we discuss the results and conclude the paper.

2 Problem statement

We study the problem of extracting and characterizing the inductive bias of learning models by finding the datasets that they generalize best. We now present a formal statement of the problem.

Let $\mathbb{X} \subset \mathbb{R}^n$ be the input space, $\mathbb{Y} \subset \mathbb{R}^m$ be the output space and $\mathbf{f}_\theta : \mathbb{X} \rightarrow \mathbb{Y}$ be the target model, where $\theta \in \Theta \subset \mathbb{R}^k$ are its parameters. Let $\ell(\mathbf{f}_\theta(\mathbf{x}), \mathbf{y})$ be the loss function of the target model.

Consider a parametric family of distributions $p(\mathbf{x}|\mathbf{y}, \mathbf{w})$, where $\mathbf{w} \in \mathbb{W} \subset \mathbb{R}^l$ are the parameters of the distribution. For a fixed label $\mathbf{y} \in \mathbb{Y}$, a sample $\mathbf{x} \sim p(\mathbf{x}|\mathbf{y}, \mathbf{w})$ is produced by the generator $\mathbf{g}(\mathbf{w})$.

We aim to find data distribution function p^* , corresponding to parameters $\mathbf{w}^*$, the solution to the following optimization problem:

$$\mathbf{w}^* = \arg \min_{\mathbf{w} \in \mathbb{W}} \mathbb{E}_{\mathcal{D}_{\text{syn}} \sim p(\mathbf{x}|\mathbf{y}, \mathbf{w})} [\mathcal{L}(\mathbf{f}_\theta, \mathcal{D}_{\text{test}})] \quad \text{s.t.} \quad \mathcal{D}_{\text{syn}} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}} \in \mathcal{U}$$

where

$$\hat{\theta} = \arg \min_{\theta \in \Theta} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_{\text{train}}} \ell(\mathbf{f}_\theta(\mathbf{x}), \mathbf{y}).$$

$\mathcal{D}_{\text{syn}}$ denotes the synthetic dataset generated by $\mathbf{g}(\mathbf{w})$. In the optimization problem, the minimum is sought not over all possible datasets, but only over those belonging to a set $\mathcal{U}$, which may vary

depending on the problem setting. In our case, $\mathcal{U}$ defines the structure of the dataset: the number of classes is specified, and for each class, $\mathcal{D}_{\text{syn}}$ contains a fixed number of samples, which is the same for every class.

Function $\mathcal{L}$ measures a model's ability to generalize by calculating the prediction error on a test set and also contains regularization terms to avoid trivial solutions. For example, prediction error can be estimated as cross-entropy or mean squared error (MSE), defined respectively as:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_{\text{test}}} \sum_{i=1}^m y_i \log([\mathbf{f}_{\hat{\theta}}(\mathbf{x})]_i), \quad \mathcal{L}_{\text{MSE}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_{\text{test}}} \|\mathbf{y} - \mathbf{f}_{\hat{\theta}}(\mathbf{x})\|^2.$$

3 Method

Our proposed approach is to meta-train a generative model to produce datasets on which the target model exhibits the best generalization performance, i.e., datasets to which it is inductively biased. In this section we describe our method in detail.

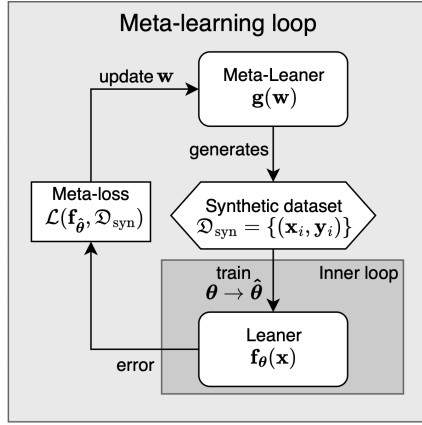


Figure 1: Meta-learning scheme

The meta-learning cycle is structured as follows. First, the generator $\mathbf{g}(\mathbf{w})$ with current parameters $\mathbf{w}$ constructs a synthetic dataset $\mathcal{D}_{\text{syn}}$ according to the predefined configuration: for each class, a fixed number of samples is generated. The number of classes and the number of samples per class are both specified as hyperparameters. Then, the resulting dataset is split into training $\mathcal{D}_{\text{train}}$ and test $\mathcal{D}_{\text{test}}$ sets.

Next, the target model $\mathbf{f}_{\theta}$ is trained on the training set $\mathcal{D}_{\text{train}}$ in the inner loop. It is important that the model be re-initialized from scratch at each meta-learning iteration. At the output of the inner loop we obtain a trained model $\mathbf{f}_{\hat{\theta}}$.

Next, the meta-loss function is calculated. It consists of two terms. The first term $\mathcal{L}_g$ measures the model's generalization quality, estimated on the test set $\mathcal{D}_{\text{test}}$. This could be, for example, MSE or Cross-Entropy. The second term $\mathcal{R}$ is the regularization term. Note that if all objects within a class are identical, the target model would easily

achieve 100% accuracy. However, this degenerate case is uninformative. The regularization term is precisely what prevents such trivial solutions. Possible choices include the average distance between objects within the same class, defined as:

$$\mathcal{R}_{\text{dist}} = -\frac{1}{C} \sum_{k=1}^C \frac{1}{n_k(n_k - 1)} \sum_{\substack{i, j \in \mathcal{I}_k \\ i \neq j}} \|\mathbf{x}_i - \mathbf{x}_j\|_2^2,$$

where C is the number of classes in the dataset, n_k is the number of objects in class k , and $\mathcal{I}_k$ is the set of indices of objects belonging to class k .

Another regularization strategy is to penalize small singular values of the generator's weight matrices, as this helps to avoid degeneracy. The corresponding regularization term is defined as:

$$\mathcal{R}_{\text{svd}} = \frac{1}{K} \sum_{k=1}^K \frac{1}{r_k} \sum_{i=1}^{r_k} (\max(\tau - \sigma_i^{(k)}, 0))^2,$$

where K is the number of linear layers in the generator, $\sigma_1^{(k)}, \dots, \sigma_{r_k}^{(k)}$ are the singular values of the k -th weight matrix and τ is a fixed threshold.

Thus, during meta-training, the meta-loss is minimized by updating the generator parameters $\mathbf{w}$, resulting in a generator $\mathbf{g}(\hat{\mathbf{w}})$ that produces datasets which are easy for the target model to generalize from. The structure of these datasets reflects the model's inductive bias.

The proposed meta-learning procedure is summarized in Algorithm 1.

Algorithm 1 Meta-learning for inductive bias extraction

- 1: Initialize generator $g(\mathbf{w})$ with random parameters $\mathbf{w}$
 - 2: **while** step < total steps **do**
 - 3: Generate synthetic dataset $\mathcal{D}_{\text{syn}} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$ with C classes and N samples per class
 - 4: Split $\mathcal{D}_{\text{syn}}$ into training set $\mathcal{D}_{\text{train}}$ and test set $\mathcal{D}_{\text{test}}$
 - 5: Re-initialize target model $\mathbf{f}_{\theta}$ from scratch
 - 6: Train $\mathbf{f}_{\theta}$ on $\mathcal{D}_{\text{train}}$ in the inner loop $\rightarrow$ trained model $\mathbf{f}_{\hat{\theta}}$
 - 7: Compute meta-loss: $\mathcal{L}_{\text{meta}} = \mathcal{L}_g(\mathbf{f}_{\hat{\theta}}, \mathcal{D}_{\text{test}}) + \lambda \mathcal{R}(\mathcal{D}_{\text{syn}})$
 - 8: Take $\mathbf{w}$ gradient step on meta-loss
 - 9: **end while**
-

Similar to previous work [12], our approach can be extended to compare the inductive biases of two models, i.e., to find a dataset that is easy for one model to generalize from but difficult for the other. In this case, both models are trained in the inner loop, and the measure of the generalization quality of the second model is subtracted when computing the meta-loss. Consequently, minimizing this meta-loss improves the generalization of the first model while degrading that of the second.

4 Experiments

To empirically validate our algorithm, we conducted computational experiments to extract the inductive bias of CNNs and RNNs. Specifically, we verify that the generative model learns to create datasets on which the target model exhibits the best generalization performance and attempt to interpret the obtained results in light of the well-known inductive biases of these architectures.

All data in our experiments are fully synthetic. In all experiments, we use a trainable tensor as the generative model, meaning that the elements of the synthetic dataset are directly optimized as parameters. The structure of the datasets is defined by a given number of classes and a fixed number of samples per class.

The meta-learning process is implemented using the learn2learn library [13]. Specifically, to enable end-to-end differentiability, we wrap the inner learner in a MAML (Model-Agnostic Meta-Learning) wrapper [14]. This allows us to clone the learner for each meta-step, train it on the generated dataset, and still keep the computational graph needed to propagate gradients from the meta-loss back to the generator. For the inner loop optimization, we use SGD. For the outer loop (meta-optimization), we use Adam [15].

The source code for all experiments is available at GitHub¹.

4.1 CNNs and 1×4 Images

We first test our method on a simple example: CNNs with a 1×2 convolution and 1×4 images.

The CNN architecture is as follows: a convolution with a 1×2 kernel and stride (1,2), followed by max pooling, and finally a linear layer that outputs class logits. In the experiment we use the following parameters: 2 classes, 20 samples per class, 7 inner loop steps, and learning rates of 0.5 and 0.05 for inner and outer loop respectively. As a measure of prediction error on the test set, we use MSE. First, we try without regularization. Then, we add regularization via the average intra-class distance with a parameter $\lambda = 0.5$.



Figure 2: Obtained dataset without regularization

¹Code at <https://github.com/intsystems/2026-Project-205>



Figure 3: Obtained dataset for average intra-class distance regularization

As can be seen in Figure 2, even without regularization, the trivial solution — where all images within a class are identical — does not fully occur. This reveals the convolutional architecture’s inductive bias towards locality: within a given class, we observe images where the first and second pairs of pixels are swapped. This indicates that for this architecture, local structure matters more than global structure.

Adding the mean intra-class distance as a regularization term makes the classes more diverse while preserving locality. The obtained dataset is presented in Figure 3. In class 1, the defining pattern is a white pixel followed by a black pixel (in that exact order) in one of the two positions, while the other position can contain anything — two black pixels, two gray, one white and one black (except for zero class patterns). Moreover, this pattern may appear either in the left or the right pair. This reflects the inductive bias towards locality: the label is determined by the local structure rather than the global one. The effect of stride (1,2) is also noticeable — class 0 contains an element with a white pixel followed by a black pixel in the middle, but this does not cause it to be misclassified as class 1.

To verify locality, we performed the following. We first split the obtained dataset into training and test sets. We then swapped the left and right parts of the images in the test set. After training the model on the original training set, we evaluated its accuracy on the swapped test set. Similarly, we trained the model on the swapped training set and evaluated it on the original test set. The results were averaged over 1000 random train-test splits. As can be seen from Table 1, the model achieves similar accuracy in both cases, confirming the locality of the data.

Table 1: Accuracy comparison on original and swapped data

Training	Test	Accuracy (%)
Original	Original	98.78 ± 8.82
Original	Swapped	98.78 ± 8.82
Swapped	Swapped	99.08 ± 7.25
Swapped	Original	99.08 ± 7.25

Thus, the structure of the obtained datasets reflects the inductive bias of CNNs toward data with local structure, which is consistent with previously known results.

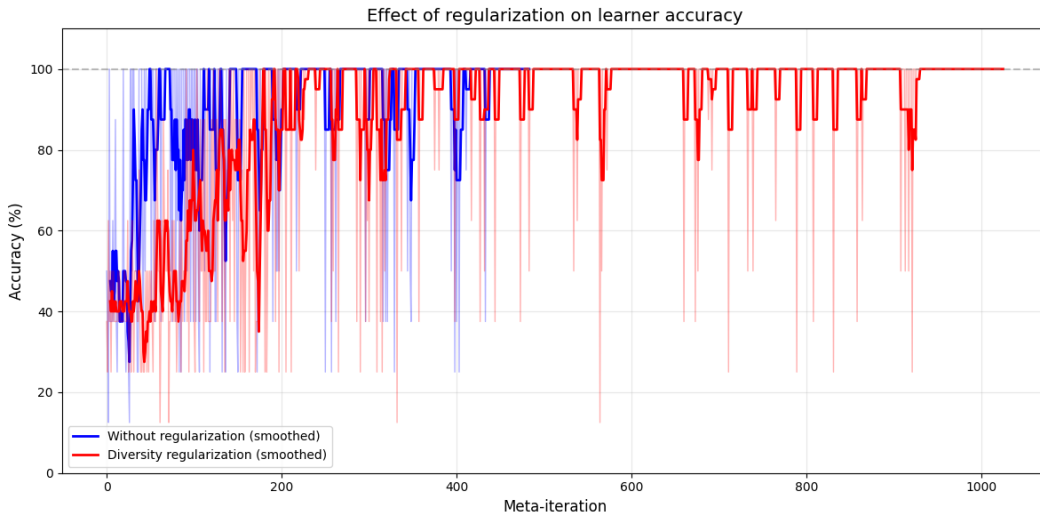


Figure 4: Comparison of meta-loss dynamics with and without regularization

4.2 CNNs and 4x4 images

Next, we conduct an experiment with CNNs on 4x4 images.

First, we consider the following CNN architecture. It consists of a single convolutional layer with a 2x2 kernel and stride 2, which reduces a 4x4 input to 4 feature maps of size 2x2. This is followed by ReLU activation and 2x2 max pooling, resulting in 4 feature maps of size 1x1. After flattening, a fully connected layer maps the 4 features to the output classes.

The experimental parameters are as follows: 4 classes, 20 samples per class, 10 inner loop steps, and learning rates of 0.5 and 0.01 for the inner and outer loops, respectively.

As meta-loss functions, we experiment with both MSE and cross-entropy (denoted CE). For regularization, we consider two strategies: the average intra-class distance (denoted Dist) and a penalty on small singular values of the generator’s weight matrices (denoted SVD). We evaluate all four combinations of meta-loss and regularization. The selected regularization parameter λ for each configuration are summarized in Table 2.

Table 2: λ for different meta-loss combinations

	Dist	SVD
CE	1.5	1.2
MSE	0.1	0.5

The obtained datasets are presented on Figures 5-8.

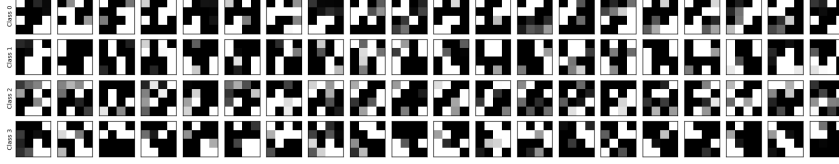


Figure 5: Obtained dataset for cross-entropy and average intra-class distance regularization

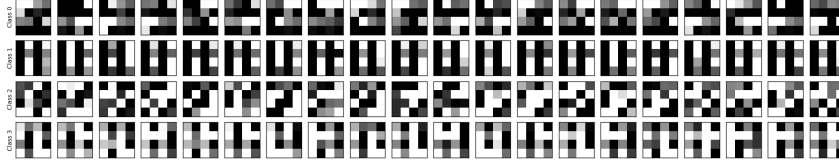


Figure 6: Obtained dataset for cross-entropy and SVD regularization

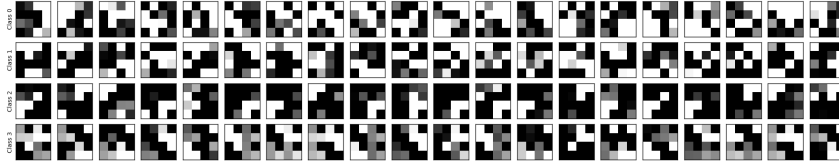


Figure 7: Obtained dataset for MSE and average intra-class distance regularization

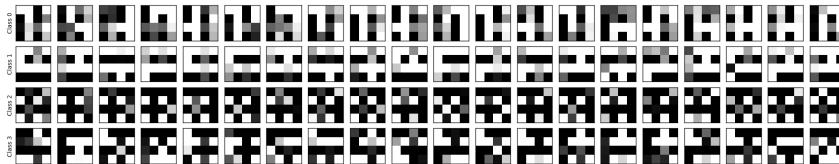


Figure 8: Obtained dataset for MSE and SVD regularization

Figure 9 compares the accuracy dependence on meta-iteration across different configurations.

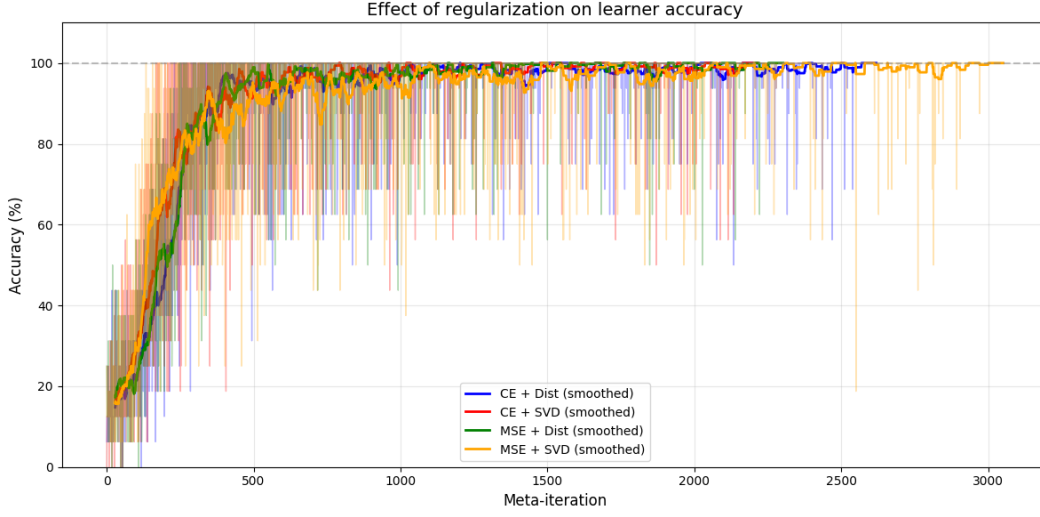


Figure 9: Comparison on different meta-loss strategies

To characterize the properties of the obtained datasets, we carry out additional analysis to verify their locality. For each configuration of the meta-loss function, we train the target model on the original training set and then evaluate its accuracy on both the original test set and a shuffled version of the test set, where the 2×2 blocks within each image are randomly permuted. The results are averaged over 1000 random train-test splits and over different seeds for shuffling. All results are summarized in Table 3.

Next, we compare our CNN with an MLP, a model that has no explicit inductive bias toward locality and is sensitive to global structure. We repeat the experiment using the same inner-loop parameters: training on the original training set and evaluating on both the original and shuffled test sets. The results are also shown in Table 3.

Table 3: Accuracy on original and shuffled data for all configurations

Configuration	Original	Seed 42	Seed 123	Seed 456	Seed 789
CE + Dist	98.87 ± 5.33	98.87 ± 5.33	98.87 ± 5.33	98.87 ± 5.33	98.87 ± 5.33
MSE + Dist	99.03 ± 5.22	99.03 ± 5.22	99.03 ± 5.22	99.03 ± 5.22	99.03 ± 5.22
CE + SVD	99.33 ± 4.62	99.33 ± 4.62	99.33 ± 4.62	99.33 ± 4.62	99.33 ± 4.62
MSE + SVD	98.96 ± 5.46	98.96 ± 5.46	98.96 ± 5.46	98.96 ± 5.46	98.96 ± 5.46
MLP	62.24 ± 14.73	78.76 ± 13.02	77.41 ± 12.97	77.61 ± 12.97	76.05 ± 13.68

As can be seen from the table, the CNN performs well even on the shuffled test set, maintaining high accuracy. This indicates locality: the model relies on the local structure within 2×2 blocks rather than on their global arrangement. In contrast, the MLP yields notably lower accuracy on both the original and shuffled test sets compared to the CNN.

These results confirm the local structure of the generated data, which aligns with the known inductive bias of CNNs.

Now we slightly modify the CNN architecture and conduct an experiment to test translation invariance.

The CNN architecture is as follows: a convolution with a 2×2 kernel and stride 1, followed by ReLU activation and 2×2 max pooling, resulting in 4 feature maps of size 1×1 . After flattening, a fully connected layer maps the 4 features to the output classes.

The obtained dataset is presented on Figure 10.

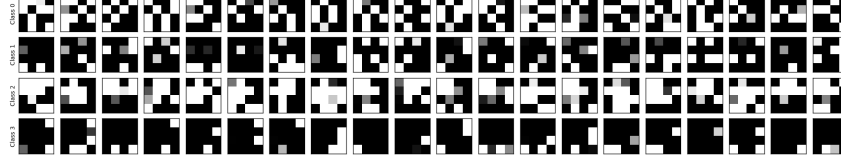


Figure 10: Obtained dataset

To evaluate translation invariance, we train the target model on the original training set and measure its accuracy on the original test set and on test sets shifted by 1 pixel horizontally, vertically, and in both directions. The results are averaged over 1000 random train-test splits and summarized in Table 4.

Table 4: Accuracy on original and shifted images

Shift type	Accuracy (%)
Original	98.78 ± 8.82
Shift horizontal (1px)	95.23 ± 10.12
Shift vertical (1px)	94.56 ± 11.34
Shift horizontal+vertical (1px)	89.12 ± 14.56

As can be observed, the accuracy on the shifted datasets exhibits only a minor drop, indicating translation invariance. This aligns with the well-known inductive bias of CNNs.

Thus, our method demonstrates strong performance on CNNs.

4.3 RNNs

Now we conduct an experiment with RNNs.

The architecture of the target model is as follows: the RNN consists of two recurrent layers, each with a hidden dimension of 32. It takes one-hot encoded sequences as input and uses the final hidden state for classification via a linear layer.

The experimental parameters are as follows: 2 classes, 20 samples per class. Each sample is a 4-letter sequence from the alphabet A, B, C, D, E, with each letter encoded as a one-hot vector. The inner loop has 10 steps, with learning rates of 0.5 and 0.005 for the inner and outer loops, respectively. We use cross-entropy as the meta-loss function. For regularization, we use a combination of the average intra-class distance and a positional diversity term that maximizes the entropy of symbol distributions at each position within a class, encouraging diversity at each sequence step:

$$\mathcal{R} = \alpha \cdot \mathcal{R}_{\text{dist}} + \beta \cdot \mathcal{R}_{\text{pos}},$$

$$\mathcal{R}_{\text{pos}} = \frac{1}{C} \sum_{c=1}^C \frac{1}{L} \sum_{l=1}^L \left(1 - \frac{H_{c,l}}{\log V} \right), \quad H_{c,l} = - \sum_{v=1}^V p_{c,l,v} \log p_{c,l,v},$$

where C is the number of classes, L is the sequence length, V is the vocabulary size, and $p_{c,l,v}$ is the empirical probability of symbol v at position l in class c .

In our experiment, the regularization parameters are $\lambda = 1$, $\alpha = 0.2$, $\beta = 0.8$.

The obtained dataset is presented in Figure 11.

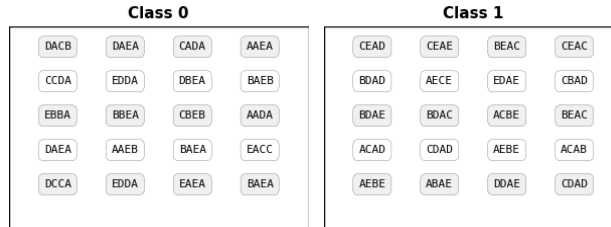


Figure 11: Obtained dataset

To verify the sequential structure of the obtained dataset, we perform the following experiment. We train the model on three different training sets: original, reversed, and randomly permuted sequences. Each trained model is then evaluated on all three test sets (original, reversed, and permuted). The results are averaged over multiple random train-test splits and presented in Table 5.

Table 5: Accuracy on original, reversed and permuted test sets

Train data	Test: Original	Test: Reversed	Test: Permuted
Original	100.00 ± 0.00	0.00 ± 0.00	50.12 ± 17.01
Reversed	0.05 ± 0.79	99.80 ± 3.57	49.39 ± 17.25
Permuted	38.88 ± 22.07	37.27 ± 19.59	38.42 ± 14.98

As can be observed, the accuracy on reversed sequences is about 0 %, which indicates that the model assigns them to exactly the opposite class. For randomly permuted sequences, the accuracy is about 50 %, which is consistent with random assignment to classes.

These results confirm the importance of order for objects in the obtained dataset, from which we conclude that the data has a sequential structure. This aligns with the well-known inductive bias of RNNs.

Thus, our method demonstrates strong performance on RNNs.

5 Conclusion

We propose a generative meta-learning framework for extracting the inductive bias of differentiable learning algorithms without prior knowledge of the input data distribution. Unlike prior work [12] that generates only labels for samples from a fixed distribution, our approach produces entire synthetic datasets via a meta-learned generative model. The resulting datasets are easy for the target model to generalize from and reflect its inductive bias.

We empirically validate our method in experiments with CNNs and RNNs and extract the well-known inductive biases of these architectures: locality and translation invariance for CNNs, and sequential structure for RNNs.

Thus, our method successfully extracts inductive bias, offering a tool for analyzing different model architectures.

References

- [1] Tom M Mitchell. The need for biases in learning generalizations. 1980.
- [2] Foster John Provost and Bruce G Buchanan. Inductive policy: The pragmatics of bias selection. *Machine Learning*, 20(1):35–61, 1995.
- [3] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- [4] Zihao Wang and Lei Wu. Theoretical analysis of inductive biases in deep convolutional networks, 2024. URL <https://arxiv.org/abs/2305.08404>.
- [5] Nadav Cohen and Amnon Shashua. Inductive bias of deep convolutional networks through pooling geometry, 2017. URL <https://arxiv.org/abs/1605.06743>.
- [6] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, November 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [7] Taiga Ishii, Ryo Ueda, and Yusuke Miyao. Empirical analysis of the inductive bias of recurrent neural networks by discrete fourier transform of output sequences, 2023. URL <https://arxiv.org/abs/2305.09178>.

- [8] Igor Dubinin and Felix Effenberger. Fading memory as inductive bias in residual recurrent networks. *Neural Networks*, 173:106179, 2024. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2024.106179>. URL <https://www.sciencedirect.com/science/article/pii/S0893608024001035>.
- [9] Akhilan Boopathy, William Yue, Jaedong Hwang, Abhiram Iyer, and Ila Fiete. Towards exact computation of inductive bias, 2024. URL <https://arxiv.org/abs/2406.15941>.
- [10] Chenglei Si, Dan Friedman, Nitish Joshi, Shi Feng, Danqi Chen, and He He. Measuring inductive biases of in-context learning with underspecified demonstrations, 2023. URL <https://arxiv.org/abs/2305.13299>.
- [11] Michael Y Li, Erin Grant, and Thomas L Griffiths. Meta-learning inductive biases of learning systems with gaussian processes. In *Fifth Workshop on Meta-Learning at the Conference on Neural Information Processing Systems*, 2021.
- [12] William Dorrell, Maria Yuffa, and Peter Latham. Meta-learning the inductive biases of simple neural circuits, 2023. URL <https://arxiv.org/abs/2211.13544>.
- [13] Sébastien M. R. Arnold, Praateek Mahajan, Debajyoti Datta, Ian Bunner, and Konstantinos Saitas Zarkias. learn2learn: A library for meta-learning research, 2020. URL <https://arxiv.org/abs/2008.12284>.
- [14] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks, 2017. URL <https://arxiv.org/abs/1703.03400>.
- [15] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>.